



Today's agenda

↳ most frequently stack → ~~hand~~?

↳ min stack

↳ longest unbalanced subsequence

↳ K stacks

Adapted → stack as queue
↳ min stack



AlgoPrep



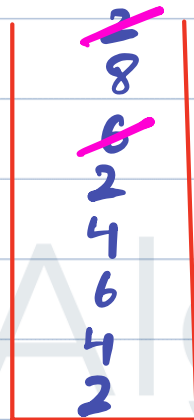
Q2) max frequency stack

↳ Design a stack like data structure supporting

1) void Push(int val) : Pushes integer on top of stack.

2) int Pop() : remove and returns the most frequent element in the stack.

Ex: 2 4 6 4 2 6 8 2 Pop() Pop() Pop()



6 → 2

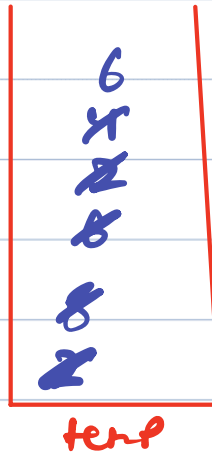
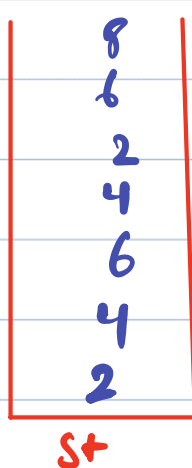
4 → 2

2 → 2

↓
2

//idea:

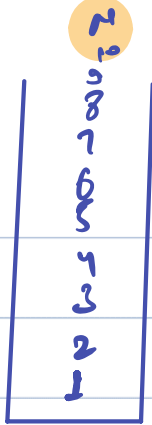
↳ Keep adding in stack and when need to Pop(), Count the freq and Pop the max freq element.



T.C: $O(n)$

S.C: $O(n)$

$N=10$

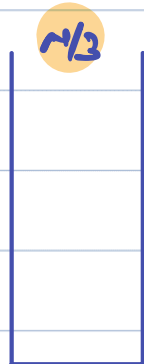


$\rightarrow O(N)$

$N=10$



$\rightarrow O(N)$



$\rightarrow O(N)$

1/idea2

2 4 6 4 2 6 8 2 POP() POP() POP()

$\frac{1}{2}$

$\frac{1}{6}$

$\frac{1}{2}$

Integer	Stack
1	2
2	4
3	6

Integer	Integer
2	2
4	2
6	1
8	1

int mf: 2 2 2 2



// Pseudo Code

```
class manfreqStack {
```

```
    HashMap< Integer, Stack< Integer >> St;
```

```
    HashMap< Integer, Integer > fmap;
```

```
    int manfreq = 0;
```

```
    void push (int val) {
```

```
        int f = fmap.getOrDefault(val, 0);
```

```
        f++;
```

```
        fmap.put (val, f);
```

$O(1)$ ←

```
        if (St.containsKey(f) == false) {
```

```
            St.push (f, new Stack<>());
```

```
            manfreq = man(manfreq, f);
```

```
        }
```

```
        St.get(f).add(val);
```

```
    }
```

```
    int Pop ( ) {
```

```
        int ans = St.get(manfreq).Pop();
```

```
        int f = fmap.get(ans);
```

```
        f--;
```

```
        fmap.put (ans, f);
```

$O(1)$ ←

```
        if (St.get(manfreq).Size() == 0) {
```

```
            St.remove (manfreq);
```

```
            manfreq--;
```

```
        }
```

```
        return ans;
```

```
    }
```

$\sim$ operations

T.C: $O(N)$

S.C: $O(N)$



```
void push (int val) {
```

```
    int b = fmap.getOrDefault(val, 0);
```

```
    b++;
```

```
    fmap.put (val, b);
```

```
    if (st.containsKey(b) == false) {
```

```
        st.push (b, new Stack<>());
```

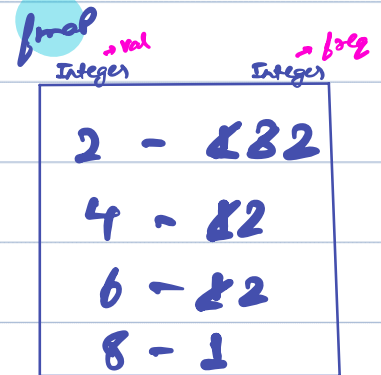
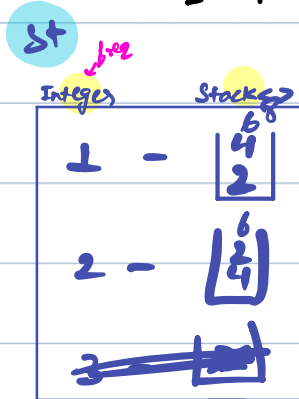
```
        manfreq = man(freq, b);
```

```
    }
```

```
    st.get(b).add(val);
```

```
}
```

2 4 6 4 2 6 8 2 pop() pop() pop()



int ans = 2

ans = 2

b = 2

```
int pop () {
```

```
    int ans = st.get(manfreq).pop();
```

```
    int b = fmap.get(ans);
```

```
    b--;
```

```
    fmap.put (ans, b);
```

```
    if (st.get(manfreq).size() == 0) {
```

```
        st.remove (manfreq);
```

```
        manfreq--;
```

```
    }
```

```
    return ans;
```

```
}
```



Q) longest unbalanced subsequence

↳ Given a string containing '(' or ')', return the longest length unbalanced subsequence.

Ex 1: (()) () () $\rightarrow$ () () $\rightarrow 5$
 $\rightarrow$ () () () () $\rightarrow 6$

Ex 2: () ()) () $\rightarrow 7$

Idea 1

↳ Create all subsequence, check if they are unbalanced and return the longest length.

T.C: $O(2^N \times N)$

Idea 2

Ex 1: ^{0 1 2 3 4 5 6 7} (()) () () $\rightarrow$ if string is balanced
 $\uparrow$
return length - 1.

Ex 2: () ()) () $\rightarrow$ if string is unbalanced
 $\downarrow$
return length.

↳ Normal Stack $\rightarrow$ Pop(), Push(), Peek(), size()

$+$
`getMin()` $\rightarrow$ min element of Stack

↳ expected T.C: $O(1)$

Ex: 10 12 14 ~~8~~ ~~16~~ ~~18~~ ~~20~~ ~~6~~ ~~15~~ POP() POP()

Idea 2

$\rho_{OP}(1)$ $\rho_{OP}(1)$ $\rho_{OP}(1)$ $\rho_{OP}(1)$
 $\frac{1}{20}$ $\frac{1}{18}$ $\frac{1}{16}$ $\frac{1}{8}$

ohne -2

min = 1086

return min +
zcm

2 min will remain
as it is.

Return min

next min: min -
den



① Push : Add in Stack (val - min) and try to update the min variable.

obs: if we are adding (-ve) in stock, min will be updated.

① Pop:

return min + xlm

2 min will remain
as it is.

Return min

nextmin: min -
gen



//Pseudo Code

Class minstack {

Stack < Long > st;

long min = Integer.MAX_VALUE;

void Push (int val) {

if (st.size() == 0) {
 min = val;
 st.push(0);
 return;
}

$O(1)$ ←

long diff = val - min;

st.push(diff);

if (val < min) {
 min = val;
}

}

void Pop() {

long rem = st.pop();

if (rem < 0) {
 int val = min;

min = min - rem;

}

else { val = min + rem; }

}

$O(1)$ ←



```
void top() {  
    long rem = st.top();  
    if (rem < 0) {  
        int val = min;  
        return val;  
    }  
    else if (int val = min + rem; )  
        return val;  
}
```

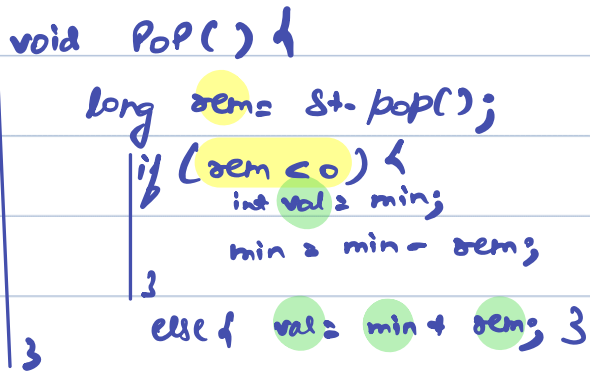
$O(1) \leftarrow$

```
int getmin() {  
    if (st.size() == 0) { return -1; }  
    return min;  
}
```

$O(1) \leftarrow$



AlgoPrep



//idea 2 POP() POP() POP() POP()

